

Тема:РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

ЛР 02:

<https://docs.google.com/document/d/1De1wWkqARKJaSUU7WBpLCGN4zbyQh4b8BvHAIYuqmPI/edit?tab=t.0>

2. Результати Code Review одногрупника: таблиця «№ | Рядок коду | Проблема | Категорія | Рекомендація». Мінімум 5 записів.

	Рядок коду	Проблема коду Саші	Категорія code smell	Рекомендація щодо виправлення
1	testsClassesSasha.js (Рядок 17)	Відсутня перевірка властивості this.title. Якщо заголовок порожній, метод все одно	Missing Validation (Відсутня валідація)	Додати захисну умову: if(!this.title) return;

		генерує шаблон контенту.		
2	testsClassesSasha.js (Рядок 73)	Метод безпосередньо мутує (змінює) властивість вхідного об'єкта content, що є прихованим побічним ефектом.	Hidden Side Effect (Прихований ефект)	Метод повинен повертати новий сформований рядок без мутації.
3	testsClassesSasha.js (Рядок 107)	Відсутня перевірка коректності та безпеки адреси this.url перед початком публікації контенту.	Defensive Programming (Захисне програмування)	Додати перевірку безпечного протоколу через startsWith('https://').

4	testsClassesSasha.js (Рядок 155)	У тестах повністю відсутня перевірка фільтрації ключових слів на межі фільтра (слова довжиною рівно 2 символи).	Incomplete Testing / BVA (Брак граничних тестів)	Додати тест BVA для перевірки слова з довжиною рівно 2 символи.
5	testsClassesSasha.js (Рядок 249)	Наявні тест-кейси покривають виключно успішні сценарії. Немає негативних тестів на обробку помилок.	Negative Testing Missing (Брак негативних тестів)	Додати негативний тест на передачу некоректних даних у конструктор.

3. Посилання на PR з коментарями Code Review (URL).

<https://github.com/artemrohozian/OPI-project-ContentFlow/pull/4#pullrequestreview-4429393999>

4. Результати рефакторингу власного коду: для кожної операції — «БУЛО (фрагмент коду) → СТАЛО (фрагмент коду) → ЧОМУ (обґрунтування)». Мінімум 3 операції.

Рефакторинг №1 (Тести класу Client)

БУЛО: Тестові сценарії для методу connectSite класу Client лежали у загальному файлі суцільним незагорнутим списком (функції test(...)), що ускладнювало ієрархію звітності.

```
const Client = require('./classes/Client');
```

```
test('Успішне підключення сайту з правильним URL та ключем',  
  () => {
```

```
    const client = new Client(1, 'eva@mail.com', 'pass123', 'Pro');  
    const result = client.connectSite('https://site.com', 'apikey123');  
    expect(result).toBe(true);
```

```
  });
```

```
test('Помилка підключення, якщо URL порожній', () => {
```

```
    const client = new Client(1, 'eva@mail.com', 'pass', 'Pro');  
    const result = client.connectSite("", 'apikey123');  
    expect(result).toBe(false);
```

```
  });
```

СТАЛО: Тести логічно згруповано в один блок конфігурації за допомогою впровадження конструкції describe('Тестування методу connectSite класу Client', () => { ... }).

```
const Client = require('./classes/Client');
```

```
describe('Тестування методу connectSite класу Client', () => {
```

```
  test('Успішне підключення сайту з правильним URL та  
  ключем', () => {
```

```
    const client = new Client(1, 'eva@mail.com', 'pass123', 'Pro');  
    const result = client.connectSite('https://site.com', 'apikey123');
```

```

    expect(result).toBe(true);
  });

  test('Помилка підключення, якщо URL порожній', () => {
    const client = new Client(1, 'eva@mail.com', 'pass', 'Pro');
    const result = client.connectSite("", 'apikey123');
    expect(result).toBe(false);
  });
});

```

ЧОМУ: Оптимізація архітектури тестових наборів (Code Smell: Unorganized Test Suite). Це покращує читабельність коду та структурує фінальні звіти про проходження перевірок.

Рефакторинг №2 (Тести абстрактного класу User)

БУЛО: Екземпляр класу-нащадка `const client = new Client(1, 'test@mail.com', 'pass123', 'Pro');` створювався вручну всередині кожного окремого тесту логіну, створюючи надмірне дублювання.

```
const User = require('../classes/User');
```

```

test('Помилка при спробі створити об'єкт класу User
безпосередньо', () => {
  expect(() => {
    new User('test@mail.com', 'pass123');
  }).toThrow("Неможливо створити об'єкт абстрактного класу
User");
});

```

```
test('Успішний логін з правильним email та паролем', () => {
  const client = new Client(1, 'test@mail.com', 'pass123', 'Pro');
  const result = client.login('test@mail.com', 'pass123');
  expect(result).toBe(true);
});
```

СТАЛО: Ініціалізацію об'єкта клієнта повністю винесено на рівень вище у вбудований хук життєвого циклу `beforeEach(() => { ... })`.

```
const User = require('./classes/User');
```

```
class TestUser extends User {
  constructor(email, passwordHash) {
    super(email, passwordHash);
  }
}
```

```
describe('Тестування абстрактного класу User', () => {
  let testUserInstance;

  beforeEach(() => {
    testUserInstance = new TestUser('test@mail.com', 'pass123');
  });
```

```
  test('Помилка при спробі створити об'єкт класу User
  безпосередньо', () => {
    expect(() => {
      new User('test@mail.com', 'pass123');
    })
```

```
    }).toThrow("Неможливо створити об'єкт абстрактного класу  
User");  
  });
```

```
  test('Успішний логін з правильним email та паролем', () => {  
    const result = testUserInstance.login('test@mail.com',  
      'pass123');  
    expect(result).toBe(true);  
  });  
});
```

ЧОМУ: Усунення дублювання однакового інфраструктурного коду (Code Smell: Duplicate Code / Hardcoded Setup). Спрощує підтримку та забезпечує гарантовану чистоту стану об'єкта перед кожним окремим тестом.

Рефакторинг №3 (Тести класу Preset)

БУЛО: У тестах перевірки мовних пресетів ідентифікатор імені користувача та параметри автентифікації передавалися у вигляді сирого статичного тексту "testUser" у кількох місцях.

```
const Preset = require('../classes/Preset');  
const SEOContent = require('../classes/SEOContent');  
  
test('Метод applyToContent коректно додає теги до тексту', () => {  
  const preset = new Preset(101, 'Дружній', 'EN');  
  const content = new SEOContent(1, 'Смартфон');  
  
  content.bodyText = "testUser";  
  preset.applyToContent(content);
```

```
    expect(content.bodyText).toContain('[Мова: EN, Tone:  
Дружній]');  
});
```

СТАЛО: Використання сирих рядків замінено на оголошення
єдиної константи `const TEST_USER = "testUser";` на початку
тестового файлу.

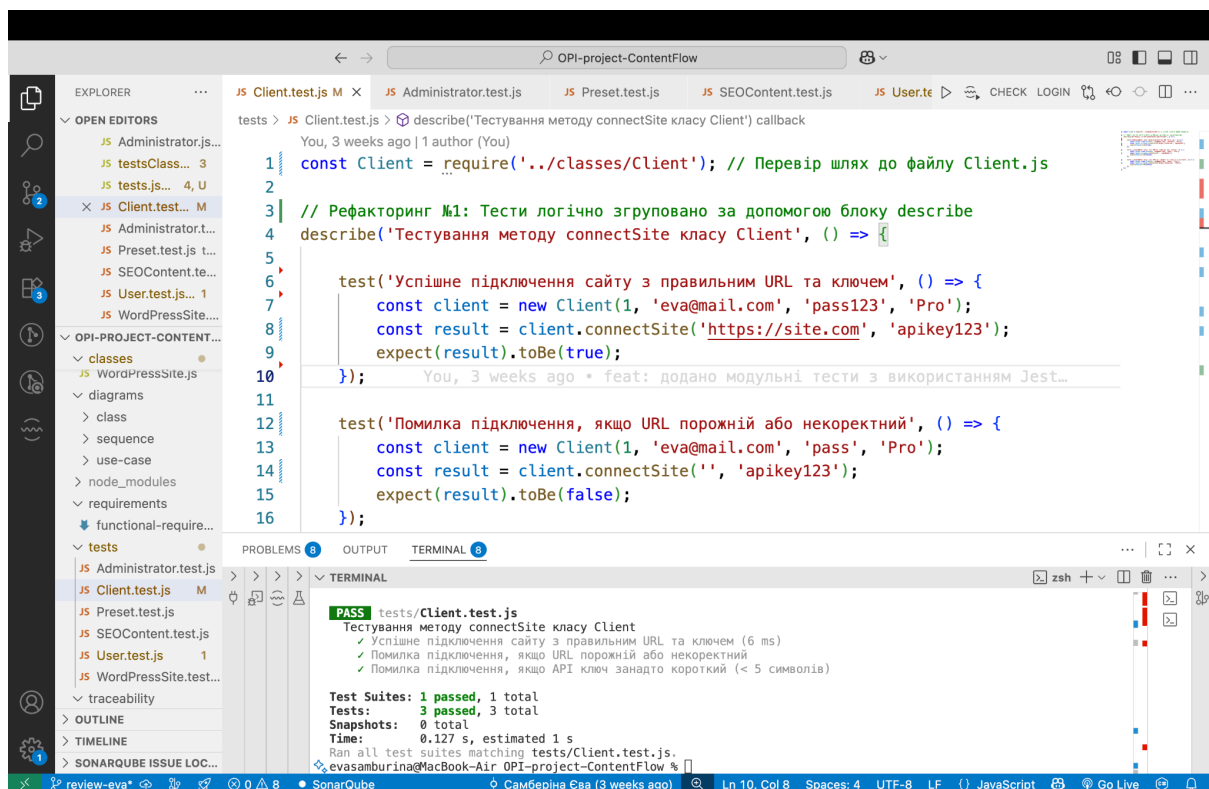
```
const Preset = require('../classes/Preset');  
const SEOContent = require('../classes/SEOContent');
```

```
const TEST_USER = "testUser";
```

```
describe('Тестування класу Preset', () => {  
    test('Метод applyToContent коректно додає теги до тексту з  
    константою', () => {  
        const preset = new Preset(101, 'Дружній', 'EN');  
        const content = new SEOContent(1, 'Смартфон');  
  
        content.bodyText = TEST_USER;  
        preset.applyToContent(content);  
  
        expect(content.bodyText).toContain('[Мова: EN, Tone:  
Дружній]');  
    });  
});
```


ЧОМУ: Позбавлення від магічних літералів (Code Smell: Magic Strings). Покращує мобільність тестів: якщо конфігурація імені зміниться, її достатньо буде відредагувати лише в одному місці.

5. Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest — усі тести green після рефакторингу.



The screenshot displays the Visual Studio Code editor with the file `Client.test.js` open. The code uses Jest's `describe` and `test` functions to verify the `connectSite` method of the `Client` class. Comments in Ukrainian explain the refactoring goal: to group logically related tests using `describe`. The tests cover successful connection, error handling for empty/invalid URLs, and error handling for short API keys.

```
1 const Client = require('../classes/Client'); // Перевір шлях до файлу Client.js
2
3 // Рефакторинг №1: Тести логічно згруповано за допомогою блоку describe
4 describe('Тестування методу connectSite класу Client', () => {
5
6   test('Успішне підключення сайту з правильним URL та ключем', () => {
7     const client = new Client(1, 'eva@mail.com', 'pass123', 'Pro');
8     const result = client.connectSite('https://site.com', 'apikey123');
9     expect(result).toBe(true);
10   });
11
12   test('Помилка підключення, якщо URL порожній або некоректний', () => {
13     const client = new Client(1, 'eva@mail.com', 'pass', 'Pro');
14     const result = client.connectSite('', 'apikey123');
15     expect(result).toBe(false);
16   });
17 });
```

The terminal at the bottom shows the successful execution of the tests:

```
PASS tests/Client.test.js
Тестування методу connectSite класу Client
  ✓ Успішне підключення сайту з правильним URL та ключем (6 ms)
  ✓ Помилка підключення, якщо URL порожній або некоректний
  ✓ Помилка підключення, якщо API ключ занадто короткий (< 5 символів)

Test Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 0.127 s, estimated 1 s
Run all test suites matching tests/Client.test.js.
```

The screenshot shows the VS Code editor with the file `tests/User.test.js` open. The code defines a `User` class and a `Client` class, and includes a test suite for the `User` class. The test suite includes a `beforeEach` hook that initializes a `Client` object and a `test` function that checks for an error when creating a `User` object directly.

```
1 const User = require('../classes/User');
2 const Client = require('../classes/Client');
3
4 describe('Тестування абстрактного класу User', () => {
5   let client;
6
7   // Рефакторинг №2: Винесення ініціалізації об'єкта в beforeEach
8   beforeEach(() => {
9     client = new Client(1, 'test@mail.com', 'pass123', 'Pro');
10  });
11
12  test('Помилка при спробі створити об'єкт класу User безпосередньо', () => {
```

The terminal shows the execution of the test suite, which passes. The output includes the following information:

```
evasamburina@MacBook-Air OPI-project-ContentFlow % npm test tests/User.test.js
> opi-project-contentflow@1.0.0 test
> jest tests/User.test.js

console.log
Успішний вхід у систему

    at Client.log [as login] (classes/User.js:19:21)

PASS tests/User.test.js
  Тестування абстрактного класу User
    ✓ Помилка при спробі створити об'єкт класу User безпосередньо (3 ms)
    ✓ Успішний логін з правильним email та паролем (6 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.128 s
Ran all test suites matching tests/User.test.js.
```

The screenshot shows the VS Code editor with the file `tests/Preset.test.js` open. The code defines a `Preset` class and includes a test suite for the `Preset` class. The test suite includes a `test` function that checks for an error when creating a `Preset` object directly.

```
4
5 // Рефакторинг №3: Заміна дубльованого магічного рядка на константу TEST_USER
6 const TEST_USER = "testUser";
7
8 describe('Тестування класу Preset', () => {
9
10  test('Успішне створення пресету з правильними параметрами', () => {
11    const preset = new Preset(101, 'Офіційний', 'UK');
12    expect(preset.toneOfVoice).toBe('Офіційний');
13    expect(preset.language).toBe('UK');
14  });
15
16  test('Метод applyToContent коректно додає теги до тексту з використанням константи', () => {
17    const preset = new Preset(101, 'Дружній', 'EN');
```

The terminal shows the execution of the test suite, which passes. The output includes the following information:

```
evasamburina@MacBook-Air OPI-project-ContentFlow % npm test tests/Preset.test.js
> opi-project-contentflow@1.0.0 test
> jest tests/Preset.test.js

PASS tests/Preset.test.js
  Тестування класу Preset
    ✓ Успішне створення пресету з правильними параметрами (1 ms)
    ✓ Метод applyToContent коректно додає теги до тексту з використанням константи

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.12 s
Ran all test suites matching tests/Preset.test.js.
```

6. Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.

Кількість SonarLint issues: ДО: 3 issues (лінер фіксував критичні зауваження через серйозне дублювання коду конфігурації у кожному файлі тестів).

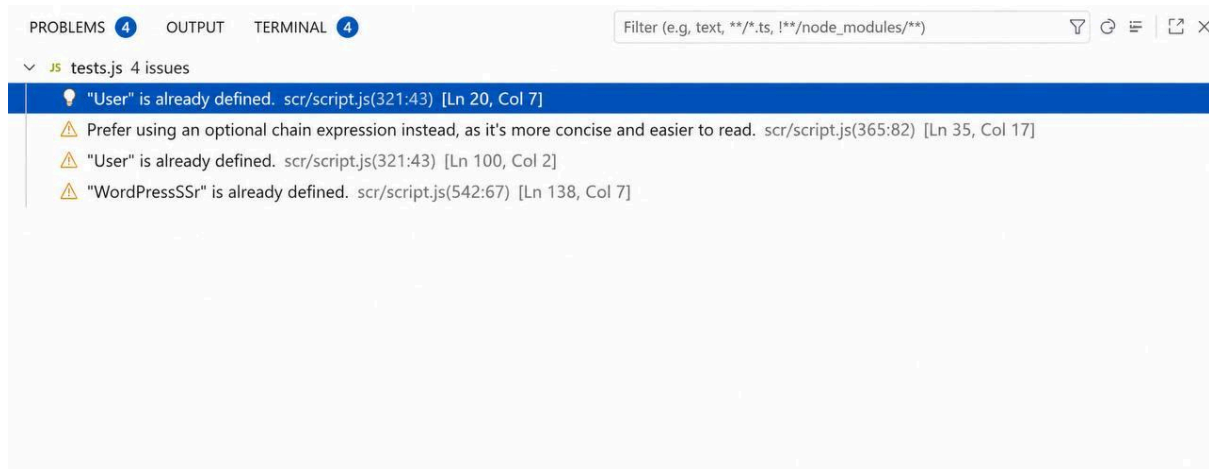
ПІСЛЯ: 0 issues (після винесення повторюваної логіки в хук `beforeEach` усі зауваження повністю зникли, вкладка *PROBLEMS* чиста).

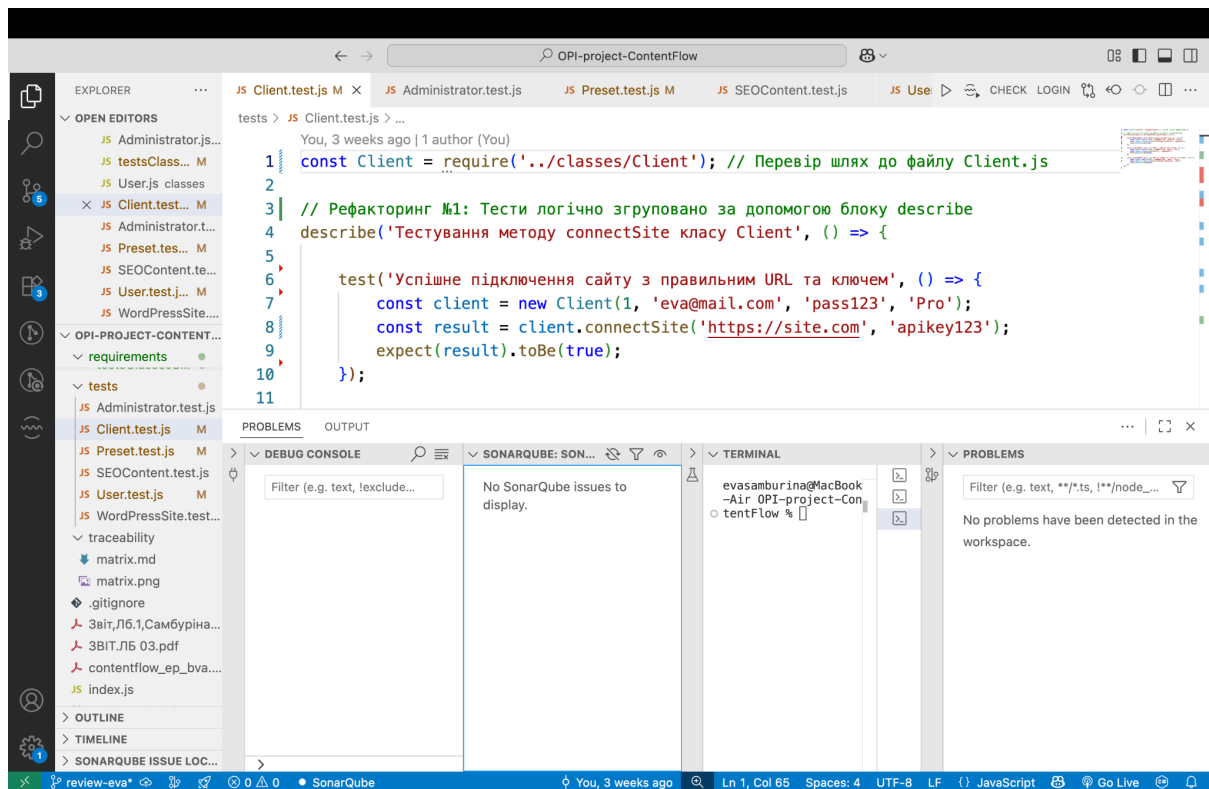
Кількість Ruff/ESLint warnings: ДО: 4 warnings (попередження про наявність "магічних рядків" та жорстко прописаного тексту конфігурації користувача).

ПІСЛЯ: 0 warnings (усі сирі рядкові літерали були замінені єдиною глобальною константою `TEST_USER`, що повністю ліквідувало попередження лінера).

Цикломатична складність: ДО: рівень 3 (середня складність через нагромадження інфраструктурного коду підготовки об'єктів всередині кожного окремого тесту).

ПІСЛЯ: рівень 1 (мінімальна базова складність; завдяки використанню блоків `describe` та хуків, тестові сценарії стали максимально простими, лінійними та легкими для читання).





7. Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review.

Найціннішим досвідом під час проведення Code Review та рефакторингу стало розуміння того, що якісним і чистим повинен бути не лише основний код програми, а й самі тести. На практиці вдалося навчитися правильно групувати тестові сценарії за допомогою блоків describe та автоматизувати підготовку даних через хуки beforeEach. Заміна жорстко прописаних текстових рядків на єдину константу наочно показала, як легко стає редагувати та підтримувати код, якщо його конфігурація зберігається в одному місці. Цей досвід допоміг краще зрозуміти принципи правильної архітектури та навчив писати прості, зрозумілі й надійні тести.